AICTE VERIFIED STUDENT REGISTER

Vibecoding Specification Pack

How to attach the right PDFs, prompt the coding agent, and keep implementation honest.

Document: 00-READ-ME Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Purpose of this pack

This pack converts the supplied PS 26263 Implementation & Tech Stack Report into a staged, MERN-aligned prototype specification. It is intended to be attached to an AI coding agent as project context. It does not assert that the software already exists.

Evidence labels used in the documents

Label	Meaning	How the agent must treat it
REPORT REQUIREMENT	The project report explicitly describes this capability or workflow.	Preserve its intent, while noting that the original implementation stack may differ.
CURRICULUM ALIGNMENT	The Sigma Batch curriculum includes the relevant learning area.	Prefer these tools where they can safely support the requirement.
PROPOSED IMPLEMENTATION	A practical design choice added to make the MERN prototype buildable.	Implement unless a conflict is identified; do not misrepresent it as an official requirement.
TBD / CONFIRM	A detail cannot be established from the source materials.	Do not guess; ask for the missing authoritative rule or use clearly labelled mock configuration.
NOT IN PROTOTYPE	A production integration or capability intentionally deferred.	Do not fake a successful real integration. Use a named mock adapter if needed.

2. File list and when to attach each PDF

File	Contains	Attach when
01_Problem_Statement_and_Product_Scope.pdf	Problem, intended outcome, user groups, in/out scope and demo flow.	Always; this defines what the product is.
02_Tech_Stack_and_Repository_Spec.pdf	Approved stack, package boundaries, folder layout and dependency rules.	Always; this prevents stack drift.
03_Authentication_and_Authorization_Spec.pdf	Eight roles, session design, permission matrix, auth endpoints and tests.	Before login and any protected page/API.
04_Role_Features_and_Button_Inventory.pdf	Screen-by-screen button/action inventory for every role.	Before frontend pages and corresponding APIs.
05_System_Architecture_and_Workflows.pdf	System diagram, modules, data flow and sequence flows.	Before repository setup and architecture work.
06_Database_and_API_Contract.pdf	MongoDB collections, fields, indexes and proposed REST API contract.	Before database models and backend routes.
07_Validation_Seat_Ledger_and_Conflicts.pdf	Validation layers, severity, seat rules, lifecycle and conflict resolution.	Before upload, admission logic and commit.
08_Security_Privacy_Testing_and_Acceptance.pdf	Security requirements, test cases, go/no-go criteria and limitations.	Throughout; mandatory before demo/deployment.
09_Vibecoding_Prompt_Pack_and_Build_Order.pdf	Copy-ready prompts and milestone acceptance gates.	When asking the agent to implement a new phase.
10_Synthetic_Demo_Data_and_Scenarios.pdf	Safe fake data schema, seed accounts, fixtures and repeatable demo scenarios.	Before creating seed data, validation fixtures or the judge demo.

3. Recommended attachment order

1. Planning only: attach 00, 01, 02 and 05. Ask the agent to produce a plan and list unresolved decisions; do not permit code generation yet.

2. Foundation: attach 01, 02, 03 and 05. Generate the skeleton, environment setup, database connection and authentication only.

3. Each feature: attach its relevant specification plus 04 (button behaviours), 06 (API/data contract), 07 (admission logic) and 08 (acceptance tests) as applicable.

4. Before review: attach 08 and ask for a defect-only audit. Do not mix a code audit with adding new features.

5. Before demo: attach 01, 04, 07, 08 and 10 to verify end-to-end scenarios and disclose all mock services.

4. Non-negotiable project invariants

- Stack: React + JavaScript + Bootstrap; Node.js + Express; MongoDB + Mongoose. No TypeScript, NestJS, PostgreSQL, Redis/BullMQ, Keycloak, Python/ML or Kubernetes unless explicitly approved as a later phase.
- Use synthetic student data in the prototype. Do not place actual APAAR, Aadhaar, DOB or other sensitive identifiers into the demo.
- Real APAAR, AICTE and DigiLocker access is not assumed. External systems must be behind adapters; demo adapters must be named and labelled mock.
- Seat capacities and policy rules must come from a clearly identified configuration/approval record. No unverified legal percentages may be hard-coded.
- Only the server authoritatively validates inputs, checks permissions, reserves seats and commits records.
- Risk scores are investigation-priority signals, not a probability of fraud and not a disciplinary decision.
- A normal QR URL is not a signed verifiable credential. Clearly label prototype QR behaviour.
- Do not report tests, integrations or deployment as successful unless actually executed and verified.

5. Definition of a finished milestone

- Acceptance criteria are written and met.
- Relevant tests were executed and their results are truthful.
- The app starts using documented commands.
- No secrets or live student information are committed.
- The agent lists changed files, new dependencies, assumptions, errors and known limitations.
- A Git checkpoint is created before moving to the next risky phase.

6. Source materials

Primary product source: PS 26263 - Implementation & Tech Stack Report, 33 pages. The source describes the verified student register, eight user types, upload/validation/certification pipeline, category-based seat ledger, conflict resolution, dashboards, consent, QR credentials, audit chain, prototype limitations and proposed production design.

Learning-scope source: Sigma12 Curriculum.pdf, 14 pages. Relevant modules include HTML/CSS, Bootstrap, JavaScript, async/await, AJAX, Git, Node.js, Express, EJS, MongoDB, Mongoose, APIs, error handling, authentication and React. The requested implementation uses React with JavaScript and Bootstrap for the frontend.

AICTE VERIFIED STUDENT REGISTER

Problem Statement and Product Scope

What problem the prototype solves, who uses it, what is included, and what must not be claimed.

Document: 01-PRODUCT Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Problem statement

Problem statement (project-level summary)
AICTE needs a way to move from aggregate admission counts to student-level admission records that can be validated, institutionally certified, monitored, and selectively verified. Incoming college data may contain formatting errors, duplicate rows, seat-category overflow, inconsistent fields, or conflicting active enrolments. The proposed platform adds an AICTE-side verified register and oversight workflow; it does not recreate APAAR, ABC or DigiLocker.

Basis: PS 26263 report, pages 1-2. This wording is a plain-language project summary, not a claim that AICTE has approved this exact implementation.

2. Product goals

- Collect admission records from college CSV files; ERP/API input is a future extension unless explicitly implemented.
- Stage incoming records so that an upload is not automatically treated as the official register.
- Apply deterministic checks and report row-level errors, warnings and pending checks.
- Track course/year/category seat occupancy using configurable rules and explicit approved-capacity data.
- Require an authorised institution head to certify before a batch is committed.
- Keep enrolment lifecycle history and flag possible cross-college conflicts for review.
- Provide role-scoped dashboards and explainable, non-punitive risk signals.
- Demonstrate student-consent verification, QR status checking and tamper-evident audit history with synthetic data.

3. Users and high-level responsibilities

Role	Main responsibility	Boundary
College Operator / Maker	Upload files, fix validation issues, track submission status.	Cannot certify or submit the final commit.
College Head / Checker	Review submission, handle warnings/conflicts and certify.	Must belong to the institution being certified.
AICTE National Officer	National monitoring, case review, escalation and reporting.	Read/decision privileges only as explicitly defined; no implicit system-admin authority.
AICTE Regional Officer	Monitor and review cases in assigned region.	Cannot view unrelated regions.
AICTE System Administrator	Manage user/role assignments and approved policy configuration.	Does not automatically gain authority to certify institutional batches.
Degree-Awarding Body	Use permitted certificate/credential issue workflow.	Only authorised student records and issuer scope.
Verifier	Request/check permitted student information.	No access to private record before consent or a separately approved legal basis.
Student	View own record, consent to sharing, see access history and credential status.	Cannot directly rewrite official enrolment records.

4. Primary end-to-end user story

1. A College Operator logs in and downloads the current demo CSV template.

2. The operator uploads a synthetic CSV. The system creates an UploadBatch and stores rows as staged data.

3. Server-side validation records row number, rule code, severity, message and resolution status.

4. The operator corrects allowed fields and requests revalidation of changed rows.

5. The batch cannot be certified while blocking errors remain. Warnings are handled according to explicit policy.

6. The College Head reviews the declaration and certifies the submission.

7. The server atomically applies seat reservations and writes the official records, history and audit events, or rolls the operation back.

8. The dashboards show updated counts; conflict detection and risk-signal processing can follow.

9. A student can approve a mock verification request; a verifier sees only permitted fields. A public QR page shows minimal status.

5. Scope

MVP / must-have	Later / explicitly deferred
Login/logout, role-based authorization, institution/region scope.	MFA, enterprise federation and production identity-provider integration.
CSV upload, staging, seven validation-layer framework, error grid and revalidation.	High-volume durable queue and production object-storage infrastructure.
Configurable seat ledger, certification, commit, enrolment history and conflict case workflow.	Official quota automation until exact rules and approvals are verified.
College and AICTE dashboards backed by persisted demo data.	Advanced analytics warehouse, national-scale performance guarantees.
Rule-based anomaly signals with reasons and evidence.	Isolation Forest or any separate ML service.
Mock consent flow, QR status and audit-chain integrity demo.	Real APAAR/ABC/DigiLocker connections, production-grade signed credentials, HSM and legal integrations.

6. Explicit non-goals and prohibited claims

- The prototype is not an official AICTE system and has no implied approval or government affiliation.
- No real APAAR, AICTE EoA, ABC or DigiLocker API availability is assumed.
- No actual student data or real sensitive identifiers are required for demonstration.
- No quota percentage, reservation rule, age rule or current law is to be asserted from this specification alone; policy owners must confirm the official source and effective academic year.
- A flagged pattern does not prove fraud. Human review and evidence are required.
- A QR that contains a URL or identifier is not cryptographic proof of authenticity.

7. Demo acceptance storyline

Demo	Expected visible result
Upload a synthetic student file	Batch ID/status, row count and progress/status field appear.
Inject invalid identifier and duplicate	Exact row errors appear and block certification until resolved.
Show configured seat capacity	Valid categories pass; capacity overflow is rejected safely.
Show cross-college active enrolment	Conflict case is created; records are not labelled fraudulent automatically.
Open AICTE console	Aggregate KPIs and role-appropriate risk queue display persisted data.
Request student verification	Student approves; verifier receives only approved fields.
Revoke demo credential	Same QR/status endpoint reports revoked.
Alter an audit event in test fixture	Integrity verification detects mismatch and reports affected sequence.

8. Product success criteria versus measured results

The original report includes proposed performance targets and an estimated scale. Treat these as design targets, not existing results. Any claim such as “100,000 rows validated in under five minutes,” “99.9% uptime,” or “85% detection” must be labelled a target until measured through reproducible tests. Do not invent measured impact figures.

9. Open decisions

- Official academic-year seat capacities and category rules: TBD from valid approvals/policy owner.
- Exact student identifier format and external validation contract: TBD; demo format checks are synthetic-only.
- Whether verifier access can ever proceed without student consent: TBD with authorised legal/product owner; default prototype policy is consent required.
- Exact deployment provider and supported MongoDB topology: TBD.
- Formal signed credential format, issuer key management and revocation design: not part of first MERN MVP.

AICTE VERIFIED STUDENT REGISTER

Tech Stack and Repository Specification

Approved MERN-aligned technologies, dependency boundaries, source tree, environment and operating assumptions.

Document: 02-TECH-STACK Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Selected implementation stack

Layer	Approved choice	Purpose / constraints
UI	React + JavaScript + Bootstrap	Role-specific screens and responsive layouts. Use plain JavaScript, not TypeScript.
UI build tooling	Vite (proposed)	Development server and production static build. Tool choice is an implementation recommendation, not explicitly listed in the curriculum.
Client routing	React Router (proposed)	Map URLs to protected role-specific pages.
HTTP	Browser fetch API (proposed)	Call REST endpoints; handle loading/error states.
Server	Node.js LTS + Express.js	REST API, business rules, authorization middleware.
Database	MongoDB + Mongoose	Document persistence and schema/model definitions.
Authentication	express-session + connect-mongo (proposed)	Server-side sessions stored in MongoDB; cookie-based browser login.
Password hashing	bcrypt (proposed)	Store only password hashes; never store plaintext passwords.
Security middleware	Helmet + express-rate-limit (proposed)	Baseline headers and request throttling; configuration must be tested.
CSV upload	Multer + a maintained CSV parser (proposed)	Accept bounded demo files and parse records safely.
QR generation	qrcode package (proposed)	Generate demo QR codes; do not treat a URL QR as a signed credential.
Tests	Node.js test runner; Supertest for API tests (proposed)	Repeatable checks of logic, APIs and permissions.
Version control	Git + GitHub	Checkpoints, branches and reviewable changes.
Local runtime	Node processes + MongoDB	Keep first setup understandable; add container orchestration only if needed.

2. Explicitly excluded from the first prototype

- NestJS and TypeScript.
- PostgreSQL, Drizzle ORM and SQL-first schema design.
- Redis, BullMQ and other durable queue infrastructure.
- Keycloak or enterprise SSO integration.
- Python/FastAPI/Isolation Forest model.
- Kubernetes, Helm, production autoscaling and multi-region disaster recovery.
- Live APAAR, ABC, AICTE or DigiLocker APIs and credentials.

Do not add substitutes silently
If a package or service seems necessary, the coding agent must explain the requirement, compare the simplest alternative, identify maintenance/security implications, and wait for approval before introducing a major technology.

3. Suggested repository layout

```
aicte-verified-register/
  client/
    src/
      app/              # routes and shared app shell
      components/       # reusable UI components
      features/         # auth, batches, ledger, conflicts, consent
      pages/            # role-specific screens
      services/         # API client functions
      styles/
  server/
    src/
      config/           # environment and database setup
      models/           # Mongoose models
      routes/           # REST route registration
      controllers/      # HTTP request/response handling
      middleware/       # auth, authorization, error handling
      services/         # business rules and transactions
      validators/       # server-authoritative input checks
      adapters/         # mock APAAR/AICTE/DigiLocker services
      utils/            # logging, hashes, helpers
    tests/
  docs/                 # implementation notes / ADRs
  .env.example
  .gitignore
  README.md
```

The exact folders may be adjusted once, before implementation, if the agent presents a clear reason. Keep routes thin; business rules belong in services and authorization in middleware plus service-level checks.

4. Dependency policy

Dependency	Decision	Required safeguard
mongoose	Use	Schema validation, indexes and explicit connection handling.
express-session + connect-mongo	Use for browser sessions	HttpOnly cookie, appropriate SameSite/Secure, expiry, session rotation after login, server-side invalidation on logout.
bcrypt	Use for password hashing	Asynchronous hash/compare; never log password values.
helmet	Use	Review defaults and test application behaviour.
express-rate-limit	Use	At minimum throttle login and public verify endpoints; avoid locking out legitimate users with simplistic global settings.
multer + CSV parser	Use for small demo uploads	File type and size limits, safe temporary storage, cleanup, malformed-file tests.
qrcode	Use for mock QR	Encode an unguessable demo verification reference, not personal data or secrets.
Any new major dependency	Approval required	Document why built-in Node/Express/Mongoose functionality is insufficient.

5. Environment configuration

```
NODE_ENV=development
PORT=5000
MONGODB_URI=replace_with_local_or_test_uri
SESSION_SECRET=replace_with_long_random_secret
CLIENT_ORIGIN=http://localhost:5173
SESSION_MAX_AGE_MS=3600000
MAX_UPLOAD_MB=5
MAX_ROWS_PER_BATCH=10000
```

The values 5 MB, 10,000 rows and one-hour session lifetime above are proposed developer defaults for a demo, not official policy or proven capacity limits. Tune them only after testing and document any change. Do not commit the real .env file.

6. MongoDB deployment requirement

The certification flow may need a multi-document transaction so the official enrolment, seat reservation, history and audit event remain consistent. MongoDB multi-document transactions require a replica set or sharded cluster. The agent must document and test a supported local replica-set setup or a suitable hosted MongoDB environment before implementing that flow. A standalone local MongoDB server must not be presented as having transaction support for these operations.

7. Operating rules

- The server is authoritative. Client-side validation improves UX but cannot replace server validation.
- Do not process an unbounded CSV or trust its filename/extension alone.
- Avoid adding a durable job queue until actual upload requirements justify it; if using an in-process demo worker, document restart/recovery behaviour.
- No performance target is accepted without an actual measured run and environment details.
- Pin dependency versions through the package lockfile; install current maintained versions at build time and record any security caveat.

8. Source alignment

Sigma Batch curriculum: frontend HTML/CSS/Bootstrap/JavaScript, async/await, AJAX, Git, Node.js, Express, MongoDB, Mongoose, APIs, error handling, authentication and React. The chosen stack keeps to this learning path. The specific packages and security architecture above are implementation recommendations, not claims that every package is directly taught by the curriculum.

AICTE VERIFIED STUDENT REGISTER

Authentication and Authorization Specification

User accounts, sessions, permissions, API boundaries and security tests for all eight roles.

Document: 03-AUTH Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Terms

Term	Meaning	Example
Authentication	Proves the user controls a valid account.	An operator logs in with email and password.
Authorization	Decides which actions and data that user may access.	The operator can upload but cannot certify.
Role-based access control (RBAC)	Permissions grouped by job role.	Only COLLEGE_HEAD may certify a batch.
Data scope	Limits which institution, region, student or case is visible.	A regional officer sees only the assigned region.
Session	Server-tracked login state presented by a browser cookie.	Logout invalidates the session server-side.

2. Proposed authentication approach

Use server-side sessions for the initial same-site React/Express prototype. Recommended packages: express-session with a MongoDB session store such as connect-mongo, plus bcrypt for password hashing. These are proposed engineering choices for the MERN version; they are not named in the source report.

1. User submits email and password over HTTPS in a production environment (localhost is allowed for development).

2. Server normalizes the account identifier, retrieves the active user, and compares the supplied password with the stored hash.

3. If valid, regenerate the session ID to reduce session-fixation risk and store only the minimum user reference/session metadata.

4. Set an HttpOnly cookie with an appropriate SameSite policy; use Secure in HTTPS deployments. Keep the actual session data server-side.

5. For each protected request, middleware loads the authenticated user and checks active status, role and data scope.

6. Logout invalidates the server-side session and clears the browser cookie.

7. Session expiration, account disablement and password reset must invalidate relevant sessions according to documented policy.

Do not mix authentication with frontend state
A hidden button or a React route guard is not a security boundary. Every protected Express route and sensitive service operation must verify identity, role, scope and business preconditions.

3. Required roles

Role enum	Account scope	Core authority
COLLEGE_OPERATOR	One assigned institution	Create/upload batches; correct staging rows; view own institution seat ledger and conflicts.
COLLEGE_HEAD	One assigned institution	Review/return/certify its institution batches; handle institution-side conflict actions.
AICTE_NATIONAL_OFFICER	National scope	View national dashboards; review/assign/escalate risk and conflict cases; view permitted audit/report data.
AICTE_REGIONAL_OFFICER	One or more assigned regions	View regional dashboards and handle cases only within assigned region.
AICTE_SYSTEM_ADMIN	System administration scope	Manage accounts/role assignments and draft/publish policy rules through maker-checker controls.
DEGREE_AWARDING_BODY	Assigned issuer organisation	View eligible records and issue/status credentials within issuer permissions.
VERIFIER	Registered verifier organisation	Create verification requests and view only results authorised for that request.
STUDENT	Own student reference only	View own record; approve/deny/revoke consent; view access history and own credential status.

4. Permission matrix

Capability	Operator	College Head	Regional	National	System Admin	Awarding Body	Verifier	Student
Upload/edit staging	Yes	Review only	No	No	No	No	No	No
Certify batch	No	Own college	No	No	No	No	No	No
View seat ledger	Own college	Own college	Assigned region summaries	National scope as permitted	Config/admin views	No by default	No	No
Resolve institution conflict	Own-side details / submit facts	Own college actions	Region case review	National escalation/review	No direct resolution by default	No	No	No
Manage users/roles	No	No	No	No	Yes	No	No	No
Change/publish seat policy	No	No	No	Review/oversight if granted	Draft and publish with separation of duties	No	No	No
Review risk case	No	No	Assigned region	National scope	No direct adjudication by default	No	No	No
Request verification	No	No	No	Legal mandate only if configured	No	No	Yes, own requests	No
Approve own consent	No	No	No	No	No	No	No	Yes
View own access history	No	No	No	No	No	No	No	Yes
Issue credential	No	No	No	Admin oversight only if approved	Policy/config only	Assigned issuer scope	No	Request/download if eligible
Public QR status	Public minimal status endpoint	Same	Same	Same	Same	Same	Same	Same

This permission matrix is the proposed implementation contract. If a source requirement does not establish a permission, the safer default is deny until the owner approves it.

5. Minimum auth endpoints

Method / path	Purpose	Access / expected outcome
POST /api/auth/login	Authenticate credentials and establish session.	Public with login rate limit; generic error on invalid credentials.
POST /api/auth/logout	Invalidate current session.	Authenticated; idempotent logout response.
GET /api/auth/me	Return current user reference, role and scope.	Authenticated; do not return password hash or secrets.
POST /api/auth/change-password	Change own password.	Authenticated; verify current password and rotate session as appropriate.
POST /api/admin/users	Create a managed user.	System admin only; allow-list role values; audit actor and changes.
PATCH /api/admin/users/:id	Change permitted user status/scope.	System admin only; prevent unsafe self-escalation and audit changes.
POST /api/admin/users/:id/password-reset	Initiate controlled reset.	System admin; never reveal an existing password or set a shared permanent password.

6. Login UI requirements

Control	Expected behaviour	Failure behaviour
Email/username field	Required, trim and normalize consistently.	Show a field-level message; never echo sensitive auth details.
Password field	Required, masked by default.	Do not log the password.
Show password toggle	Temporarily switch input visibility.	Must not send data to server until form submission.
Sign in	Submit once; show loading state.	Prevent duplicate submissions; show generic invalid-credentials message.
Forgot password	Only display if recovery is actually implemented.	Do not show a dead button. Demo may direct user to the documented admin reset process.
Sign out	Invalidate server session and return to login.	If network fails, show honest status; do not claim logout succeeded without server confirmation.

7. Security requirements

- Hash passwords with a suitable password-hashing library; never encrypt a password for later recovery and never store plaintext.
- Use a server-side session store, rotate session ID on login, set secure cookie options, configure expiration and invalidate sessions on logout.
- Use CSRF protection for cookie-authenticated state-changing requests. The coding agent must select and document a maintained implementation compatible with the app.
- Use strict CORS only if frontend/backend origins differ; do not use wildcard origins with credentialed requests.
- Rate-limit login and public verification endpoints; implement safe error messages and audit security-relevant actions.
- Do not permit self-service role selection or privileged public registration.
- Create demo users with synthetic accounts and hashed password fixtures; never commit production credentials.
- Disable account access when a user is inactive or disabled, even if a session cookie still exists.

8. Mandatory auth tests

- Valid login succeeds; wrong password and unknown user fail with no account enumeration.
- Unauthenticated access to protected endpoint returns 401.
- Authenticated user without permission receives 403.
- College A cannot read or mutate College B records by changing IDs in URL/body.
- Regional officer cannot access outside assigned region.
- Operator cannot call certification endpoint directly.
- Student cannot approve consent for another student.
- Disabled user and expired session are rejected.
- Logout invalidates the session.
- Login throttling and CSRF protection have explicit tests.

AICTE VERIFIED STUDENT REGISTER

Role Features and Button Inventory

Detailed screen/control specification: purpose, preconditions, action, resulting state and permission boundary for every proposed button.

Document: 04-ROLE-UX Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. How to read this document

The original report defines the eight user types and broad workflows, but it does not prescribe exact button captions. The labels and button inventory below are PROPOSED UI requirements to make the MERN prototype implementable. Preserve these behaviours unless the project owner approves a change. A button must never imply a feature that is not implemented.

Button rule
Every state-changing button must have a loading state, success feedback, failure feedback, disabled/precondition state and backend authorization. Hiding a button does not replace server-side permission checks.

2. Shared public and authenticated controls

Screen / button	Action and conditions	Result / error behaviour
Login: Sign in	Validate fields; submit credentials once.	On success navigate by role. On failure show generic error; never expose server stack trace.
Login: Show password	Toggle only current password input visibility.	No server request.
Global: Sign out	POST logout endpoint; clear client user state after server response.	Return to login; if server fails, display status rather than claiming confirmed logout.
Global: Refresh	Reload current page data from authorised APIs.	Keep filters if practical; show retryable error.
Global: Search / Filter / Clear filters	Filter already-authorised rows by allowed fields.	Must not widen the caller's server-side scope.
Public verify: Scan QR	Request camera permission and decode a QR payload.	If permission denied, allow image/manual reference path only if implemented.
Public verify: Upload QR image	Accept an image under a configured size/type limit and decode.	Do not store image longer than necessary; reject malformed files.
Public verify: Verify	Call minimal public status endpoint for the credential reference.	Show VALID / REVOKED / INVALID-OR-TAMPERED only when supported by actual checks; avoid personal-data disclosure.
Public verify: Clear	Clear QR payload/result from UI.	No data mutation.
Global: Export	Download current permitted dataset/report.	Enforce scope and field masking on the server; log sensitive exports where appropriate.

3. Role 1 - College Operator (Maker)

Default scope: exactly one assigned institution. The operator can prepare and correct staged data, but cannot certify or commit official records.

Screen / button	Precondition and action	Expected result / blocked case
Dashboard: Upload batch	User has COLLEGE_OPERATOR role and assigned institution.	Open upload wizard; no access to another institution by URL manipulation.
Upload wizard: Download template	Download current demo template version.	CSV template and version note download; do not imply official live template sync.
Upload wizard: Choose file	Select CSV allowed by configured type/size.	Show file name and size; reject disallowed file and malformed extension/content.
Upload wizard: Remove file	Before submission only.	Clear selected file from UI.
Upload wizard: Pre-check	Run basic client checks; server still validates authoritatively.	Show missing columns/format issues without treating client checks as final verification.
Upload wizard: Submit upload	Valid file selection and allowed upload size.	Create batch and show batch ID/status; duplicate submission must be handled idempotently or explicitly rejected.

Screen / button	Precondition and action	Expected result / blocked case
Batch list: View batch	Batch belongs to user's institution.	Show status, counts, timestamps and permitted metadata.
Batch detail: Refresh status	Batch is visible to caller.	Fetch latest batch counts/progress.
Issues grid: Filter severity	Rows returned are already institution-scoped.	Filter Error/Warning/Pending/Info without hiding unresolved blockers from batch summary.
Issues grid: Search rows	Search permitted row fields.	Do not send raw sensitive ID to analytics/logs.
Issues grid: Edit row	Batch is uncommitted and user has edit permission.	Open editable allowed fields; immutable/internal fields stay read-only.
Issues grid: Save correction	Validate field permissions and persist correction to staging only.	Record correction metadata; do not change official registry.
Issues grid: Cancel edit	Discard unsaved local changes.	Return to read mode; confirm only if unsaved edits would be lost.
Issues grid: Revalidate changed rows	Saved corrections exist.	Re-run relevant checks; preserve previous issue history if required by audit design.
Issues grid: Export issues	Batch belongs to institution.	Download issues CSV with row numbers/rule codes; mask sensitive fields.
Seat ledger: View	Institution scope matches caller.	Read-only category capacity/filled/remaining counts from configured rules.
Conflicts: Open case	Case is assigned/visible to institution.	Show only permitted case fields; allow institution-side response/action only.
Batch detail: Mark ready for review	No blocking errors remain; required warning dispositions are recorded.	Set status to READY_FOR_REVIEW; this is not certification and must not commit records.
Certification control	Not available to operator.	No button and API returns 403 if directly invoked.

4. Role 2 - College Head / Nodal Officer (Checker)

Default scope: assigned institution only. The head is the maker-checker approver for an institution batch.

Screen / button	Precondition and action	Expected result / blocked case
Dashboard: Pending submissions	Load batches for assigned institution.	Show awaiting review, returned and certified counts.
Batch review: Open summary	Batch belongs to assigned institution.	Show row counts, error/warning dispositions, seat ledger impact and declaration.
Batch review: View issue details	Inspect unresolved and resolved issues.	Show evidence and clear rule descriptions; no editing of raw official fields by default.
Batch review: Return for correction	Batch not committed. Require a reason.	Set RETURNED_FOR_CORRECTION and notify operator in-app; no commit.
Batch review: Accept warning disposition	Only for warning types whose policy explicitly allows a documented disposition.	Store reason/actor/time; never suppress blocking errors or invent policy.
Batch review: Declaration checkbox	User confirms the displayed declaration before certification.	Required confirmation before enabling certify.
Batch review: Request OTP / verify OTP	Only if OTP is implemented for the prototype.	If not implemented, do not render a fake functioning OTP control; use clearly labelled mock confirmation only.
Batch review: Certify and submit	Correct institution, authority valid, no blocking errors, policy-required warnings resolved.	Server rechecks state and permissions, then commits safely; return conflict/validation error if conditions changed.
Batch review: Cancel	Before submission.	Return without mutation.
Conflicts: View case	Case within own institution.	Show other-institution identity only to minimum scope allowed.
Conflicts: Submit resolution	Choose permitted state/action and enter justification.	Record case event; update enrolment/seat ledger atomically if required.

Screen / button	Precondition and action	Expected result / blocked case
Reports: Export submission summary	Assigned institution only.	Download summary with masked sensitive fields.
Audit: View own submissions	Permission is limited to institution's actions.	Read-only timeline of changes/certification.

5. Role 3 - AICTE National Officer

Screen / button	Precondition and action	Expected result / blocked case
National dashboard: Refresh	National officer role.	Reload national aggregate metrics.
Filters: Academic year / state / status	Use explicitly approved filter values.	Update charts and tables; raw identifiers remain masked by default.
Map/KPI: Drill down	Click a state, institution or course.	Navigate one level deeper within national scope; preserve scope breadcrumbs.
Risk queue: Open case	Case visible in national scope.	Show score, reasons, evidence references, model/ruleset version and history.
Risk queue: Assign reviewer	Select authorised officer and record assignment.	Create assignment event and notification; reject invalid/out-of-scope assignee.
Risk case: Add note	Officer authenticated and case visible.	Append note with actor/time; do not overwrite the evidence.
Risk case: Mark reviewed	Only after required note/review fields are supplied.	Set review state; this does not imply fraud finding.
Risk case: Mark false positive	Requires reason.	Record feedback for future tuning; do not delete original signal.
Risk case: Escalate	Case open and reason supplied.	Create escalation event and notify next responsible authority.
Conflict monitor: Open/escalate	Case within national permissions.	Show conflict history and SLA status; no silent direct modification of college data.
Reports: Export national report	Only fields allowed for national role.	Export aggregated/masked information; record export event if sensitive.
Audit explorer: Verify chain	Audit service available.	Show pass/fail, affected sequence and verification time; never claim pass if not actually checked.
Policy controls	Not a routine national-officer button unless separately granted.	National monitoring role does not automatically grant system-admin rule-publishing powers.

6. Role 4 - AICTE Regional Officer

Screen / button	Precondition and action	Expected result / blocked case
Regional dashboard: Refresh	Officer has one or more assigned region IDs.	Return metrics for assigned regions only.
Filters: institution/course/year	All filters are constrained by server-side region scope.	Cannot reveal other regions by editing query parameters.
Institution row: Open details	Institution belongs to assigned region.	Show allowed aggregate and case details; identifiers masked by default.
Risk queue: Open / assign / add note	Case belongs to assigned region.	Support regional review workflow; assignee must be within permitted scope.
Risk queue: Mark reviewed / false positive	Reason required where relevant.	Record review; do not create an automatic penalty or fraud determination.
Conflict monitor: Open / escalate	Conflict falls within assigned region.	Read case and escalate to national level when required.
Regional report: Export	Filters constrained to assigned regions.	Produce report with no unrelated regional data.
National map: View all regions	Not permitted.	The UI should not show unrestricted national detail; direct API attempts return 403.

7. Role 5 - AICTE System Administrator

Screen / button	Precondition and action	Expected result / blocked case
Users: Create user	Required fields valid; role allow-listed; institution/region scope supplied when required.	Create inactive or invitation-pending account according to implemented onboarding; audit creator.
Users: Save changes	Validate unique email, valid role and scope.	Persist allowed changes; prevent user from assigning themselves higher privilege through a crafted request.
Users: Disable / re-enable	Admin confirms action and provides reason.	Disabled users cannot authenticate or continue protected sessions.
Users: Initiate password reset	User exists and reset mechanism is implemented.	Generate controlled reset workflow; never reveal/set/retrieve existing password.
Policies: Create draft	Rule set has academic year/version and data source metadata.	Save draft only; does not affect active validation.
Policies: Validate rules	Run schema/consistency tests on draft.	Show invalid config and test results; block publication on errors.
Policies: Compare versions	Select two stored versions.	Show differences and metadata/source notes.
Policies: Submit for approval	Draft valid and authored by current user.	Record request for a different authorised approver.
Policies: Approve/publish	Approver differs from rule author and has permission.	Publish a new version and retain previous version; audit actor/source/effective date.
Audit explorer: Verify chain	Audit store available.	Run integrity verification and display actual result.
System settings: Save	Only explicitly documented demo settings.	Do not expose secrets in UI or logs; every security-relevant change is audited.
Direct batch certification	Not permitted by this admin role alone.	Return 403 unless a distinct, explicitly approved additional role is assigned.

8. Role 6 - Degree-Awarding Body

Screen / button	Precondition and action	Expected result / blocked case
Certificate workspace: Search eligible student	Query restricted to issuer's assigned institution/records and permitted fields.	Return masked matching records only; no open-ended national student search.
Student row: View permitted details	Record is in issuer scope.	Show the minimum fields required to issue the credential.
Credential: Generate	Student record is eligible and issuer authorised.	Create a prototype credential reference/status; record issuer and timestamp.
Credential: Download QR/certificate	A generated credential exists.	Download a clearly labelled demo credential; do not claim cryptographic authenticity unless signed validation is implemented.
Credential: View status	Credential belongs to issuer scope.	Show active/revoked/invalid state from persisted data.
Credential: Revoke	Issuer has explicit authority; reason required.	Revoke only its own permitted credential and record audit event.
Certificate list: Export	Only fields required for the issuer's purpose.	Apply masking and authorization before export.

9. Role 7 - Verifier

Screen / button	Precondition and action	Expected result / blocked case
Verifier onboarding: Submit organisation	MVP may use a seeded, pre-approved organisation; do not claim real KYC integration.	Pending/approved state recorded according to prototype workflow.
New request: Select student	Enter permitted synthetic student reference.	Do not expose record details merely because a reference is known.
New request: Select purpose	Choose one of documented demonstration purposes.	Purpose is required and persisted.
New request: Select fields	Choose only allowed fields for that request type.	Server enforces allow-list; sensitive fields cannot be added by tampering with request body.
New request: Set expiry	Choose a permitted duration from configured options.	Expired request cannot be used later.
New request: Submit	Verifier account active and input valid.	Create pending request and notify student through demo notification UI.
Request detail: Refresh status	Request belongs to caller.	Show pending/approved/denied/expired/revoked status.
Result: View approved result	Consent is valid or explicitly configured legal basis exists. Default MVP: consent required.	Return only approved fields and result metadata.
Request: Cancel	Only own pending request.	Cancel pending request; no access to student data.
Bulk verify	Not included by default.	Do not render or enable unless legal basis, role scope and safeguards are approved.

10. Role 8 - Student

Screen / button	Precondition and action	Expected result / blocked case
My record: View	Authenticated student is mapped to a studentRef.	Show own enrolment and status only; no student search endpoint.
My record: Request correction	Select an allowed correction type and explain evidence.	Create correction request; official record remains unchanged until reviewed.
Correction: Submit request	Required explanation and any permitted evidence present.	Record request status and notify responsible institution user.
Consent inbox: Open request	Request refers to this student and has not expired.	Show requester, purpose, fields, expiry and prior status.
Consent: Approve	Request still pending; fields/purpose are displayed.	Record consent with approved fields and expiry; generate a signed consent artefact only if actual signing exists; otherwise label mock.
Consent: Deny	Request pending.	Record denial; verifier receives no requested private data.
Consent: Revoke	Consent is active and revocation is permitted.	Mark revoked; future access checks fail and access history records the event.
Access log: View details	Student's own access log only.	Show verifier, timestamp, purpose and fields disclosed; avoid exposing internal secrets.
Credential: Generate/download	Credential exists or issuance is enabled for student.	Provide QR/download with status and limitations stated.
Credential: Refresh status	Credential reference belongs to student.	Display current status from verification service.
Official record: Edit directly	Not permitted.	Use the correction-request workflow; direct mutation must return 403.

11. Shared button-state rules

- Disable double-submit while a mutation is pending.
- Require confirmation for destructive/high-impact actions such as user disablement, credential revocation, certification and rule publication.
- Display the exact blocking reason when a button is disabled.
- After mutation, refresh server state rather than trusting client-side guesses.
- Do not show controls the role cannot use; still enforce authorization on the server.
- Use plain-language errors with a field/row reference, for example: “Row 214: identifier format is invalid.”
- Do not create a button for a feature that is not wired to a real API or clearly labelled demo action.

12. Button/API traceability

Before coding a button, the agent must map it to: (a) role and scope, (b) preconditions, (c) API method/path, (d) server service, (e) database mutation/read, (f) audit event if relevant, (g) success UI state, and (h) failure tests. Buttons without a complete mapping are not ready to implement.

AICTE VERIFIED STUDENT REGISTER

System Architecture and Workflow

Component boundaries, data flow, key sequences and the limits of the prototype architecture.

Document: 05-ARCH Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Architecture overview

The architecture intentionally uses a modular Express application with separate services/modules, not a fleet of microservices. It follows the revised MERN stack and omits the original report's PostgreSQL, Redis/BullMQ, Keycloak, Python ML and Kubernetes components from the first prototype.

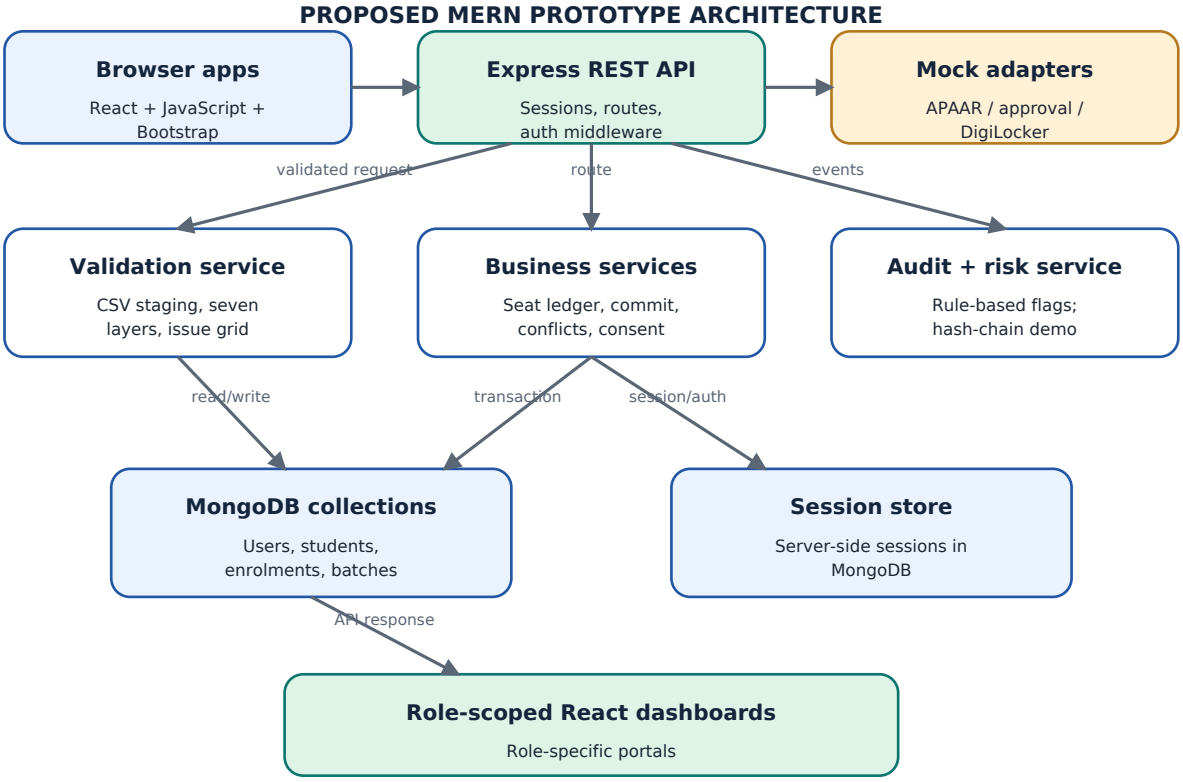


Diagram note: this is a logical view, not a claim that the implementation has already been built. Arrows show major responsibilities, not every individual HTTP call.

2. Component responsibilities

Component	Owns	Must not do
React client	Screens, field-level feedback, navigation, presentation of backend state.	Make authoritative seat decisions, trust client role data, or store secrets.
Express routes/controllers	HTTP parsing, input shape validation, invoking service methods and consistent responses.	Contain all domain logic in route handlers or bypass authorization.
Authentication middleware	Resolve session/user, check account status and required role/scope.	Trust an arbitrary role or institution ID from the request body.
Batch/ingestion service	Accept file, create batch, parse CSV safely, write staged records and statuses.	Insert incoming rows directly into the official register.
Validation service	Run deterministic L1-L6 checks and record L7 risk signals.	Treat risk flag as fraud proof or call real external APIs without access.
Seat-ledger service	Reserve/release capacity consistently with enrolment mutations.	Perform an unprotected read-then-write race-prone count.
Registry/commit service	Apply certified records and version/history/audit effects.	Partially commit records if seat reservation fails.
Conflict service	Detect and resolve potential cross-college active-enrolment conflicts.	Auto-accuse or delete the record based on a match alone.
Consent/verification service	Enforce requested fields, purpose, expiry and student approval.	Return private student data before an authorised basis exists.
Mock adapters	Simulate external data contracts in tests/demo.	Claim live integration, real API success or real identity verification.
MongoDB	Persist users, students, enrolments, batches, capacity, conflicts, consent and audit.	Be accessed directly from the frontend.

3. Upload and commit sequence

1. React submits a CSV file to POST /api/batches after client-side pre-check.

2. Express authenticates the operator and derives the institution scope from the session, not from a trusted client-supplied field.

3. Ingestion service validates file limits, creates a batch and parses into staged row records.

4. Validation service writes row-specific issues with severity and rule codes. A mock external adapter may return verified, mismatch or pending demo outcomes.

5. Operator fixes allowed fields and asks to revalidate changed rows.

6. The College Head reviews the batch and certifies only after blocking errors are cleared and required warning dispositions are present.

7. Commit service re-checks permissions, batch state, policy version and capacities inside the commit operation. It updates enrolments and seat counters consistently and records history and audit entry.

8. If any required update fails, the transaction rolls back. The batch remains uncommitted with an actionable error state.

9. Dashboard reads updated data. Risk scoring and notifications are later tasks; a prototype can trigger rule evaluation after commit without promising a durable queue.

4. Conflict resolution sequence

Actor/system	Action	Guardrail
Conflict detector	Find apparent active enrolments for same student across colleges/academic year.	Candidate match only; not a fraud conclusion.
Institution A/B users	Review their own enrolment side and provide allowed state updates/evidence.	Only see permitted minimum details about the other institution.
Conflict service	Validate transition and release/reserve seat as appropriate.	Do not close the case if required resolution conditions are missing.
Audit service	Record state change, reason, actor and timestamp.	Preserve historical events; no silent replacement of history.
AICTE officer	Review/escalate unresolved case.	Escalation does not itself determine guilt.

5. Consent and QR sequence

1. Verifier creates a request with student reference, purpose, requested fields and expiration.
2. Student portal shows the complete request and allows approve or deny.
3. Server stores the decision and checks consent/expiry on every result retrieval.
4. Verifier receives only fields that were approved.
5. Credential/QR demo points to a verification reference. The verifier checks the stored demo status.
6. Revoke updates status; an altered/malformed reference returns a generic invalid result. Do not label an unsigned QR as signed or cryptographically authenticated.

6. Synchronous versus background processing

For the first prototype, keep the data volume bounded and document the behaviour if the server restarts during validation. If a lightweight in-process worker is used, record status in MongoDB and mark unfinished jobs retry-required on startup. A durable queue (for example Redis/BullMQ) is a later scale upgrade, not part of the required stack. Do not claim reliable large-scale background processing without durable job recovery and load testing.

7. Deployment shape

- Development: React dev server, Express API, and a supported MongoDB local replica-set or test deployment when transaction flows are enabled.
- Keep frontend and backend origins explicit; configure CORS and cookie behaviour only if they are cross-origin.
- Prototype deployment can be a conventional Node frontend/API hosting setup plus MongoDB hosting; exact provider is TBD until chosen and tested.
- Production architecture in the original report (Kubernetes, advanced key management, HSM, multi-region DR and government cloud) is not delivered by this MVP.

8. Observability minimum

- Log request ID, route, status code, duration and actor reference where safe.
- Never log passwords, session IDs, tokens, full student IDs, full DOB or raw consent payloads.
- For failed commit, record batch ID and safe error code; do not print secret values.
- Expose health endpoints for the app and database connection status.
- Keep enough telemetry to reproduce a test failure and report actual processing times.

AICTE VERIFIED STUDENT REGISTER

Database Schema and API Contract

Proposed MongoDB models, identifiers, indexes and REST endpoints for the MERN prototype.

Document: 06-DATA-API Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Contract status

Important
The schemas and endpoints in this document are proposed MERN implementation contracts, not existing APIs or official AICTE endpoints. The code agent should implement them consistently and update this document if a reviewed change is approved.

2. Data modelling rules

- Use an internal random studentRef/UUID-like identifier as the platform reference. Do not make an external identifier the global primary key.
- Keep Student separate from Enrolment: the same student may have multiple enrolment histories.
- Keep incoming UploadBatch/StagingRow separate from official Enrolment until certification and commit.
- Keep SeatLedger records unique by course, academic year and bucket scope according to the approved demo model.
- Keep policy/rule versions and source metadata; do not embed unverified quota constants in code.
- Use timestamps, actor references and explicit state transitions for sensitive changes.
- Use synthetic student identifiers only. The prototype must not store real APAAR/Aadhaar details.

3. Proposed collections and fields

Collection	Proposed key fields	Important constraints
users	emailNormalized, passwordHash, role, status, institutionId?, regionIds[], studentRef?, issuerOrgId?, createdAt, updatedAt	Unique normalized email; role enum; passwordHash excluded from all API responses.
institutions	institutionCode, name, stateCode, regionId, status	Unique institution code; region linkage is authoritative for scope filtering.
courses	institutionId, courseCode, programName, level, academicYear, approvalRef, approvedIntake, approvalFlags	Index institution/year/course; approved data needs explicit source/evidence in fixture.
students	studentRef, displayName, syntheticExternalId?, externalLinkStatus, createdAt, updatedAt	Unique studentRef; external IDs are mock/synthetic only. Do not log sensitive fields.
enrolments	studentRef, institutionId, courseId, academicYear, yearOfStudy, bucketCode, state, admittedOn, version	Indexes for student/year/state and course/year/bucket/state; do not block legitimate case detection with an overly strict student-year uniqueness rule.
seatLedgers	courseId or scopeId, academicYear, bucketCode, capacity, filled, policyVersion, updatedAt	Unique configured scope/year/bucket; capacity >= 0; 0 <= filled <= capacity invariant.
seatPolicies	academicYear, version, status, buckets[], sourceReference, authoredBy, approvedBy, effectiveAt	Draft vs published version; maker-checker separation; no default real quota percentages.
uploadBatches	batchId, institutionId, academicYear, sourceFileName, fileHash, status, counts, createdBy, certifierId?, idempotencyKey?, timestamps	Batch ID unique; upload replay policy explicit; fileHash is not an authentication credential.
stagingRows	batchId, rowNumber, rawSafeFields, parsedFields, validationStatus, lastRevalidatedAt	Unique batchId+rowNumber; purge/retention of raw upload data is configurable and documented.
validationIssues	batchId, rowNumber, ruleCode, severity, messageCode, resolutionStatus, resolutionReason?, createdBy?, resolvedBy?, timestamps	Index batch+row+severity; use safe messages and rule codes.
conflicts	conflictId, studentRef, enrolmentRefs[], conflictType, status, assignedRegionId, dueAt?, resolutionType?, resolutionReason?, timestamps	Do not expose unmasked identifiers in national lists; support case history.
consents	consentId, studentRef, verifierId, purpose, requestedFields[], approvedFields[], status, expiresAt, approvedAt?, revokedAt?	Access every request through status/expiry checks; no field beyond approvedFields.
verificationRequests	requestId, studentRef, verifierId, consentId?, purpose, requestedFields[], status, expiresAt, resultReference?, createdAt	Verifier may read own request only; student sees own requests.
credentials	credentialId, studentRef, issuerOrgId, credentialType, status, issuedAt, revokedAt?, verificationMode	Prototype QR must specify url-reference/mock mode; do not call unsigned token a signed credential.
auditLogs	sequence, timestamp, actorRef, action, entityType, entityRef, eventPayloadHash, previousHash, hash, reason?	Append-only by application convention, sequential integrity checks; not immutable against a fully privileged database attacker.
notifications	notificationId, recipientRef, type, safeMessage, readAt?, relatedEntityRef, createdAt	No secrets or full sensitive identifiers in notification text.

4. Database invariants

- A college operator can read/write only batches and staging rows assigned to their institution. Do not rely on frontend filters.
- A batch transitions through documented states such as UPLOADED, VALIDATING, NEEDS_CORRECTION, READY_FOR_REVIEW, RETURNED_FOR_CORRECTION, CERTIFIED/COMMITTING, COMMITTED or FAILED. Use one agreed enum and transition table.
- No commit while blocking errors exist. Re-check at certification/commit time, because data or policy may have changed after validation.
- Seat reservation and enrolment commit must be transactionally consistent. Use a MongoDB replica set/sharded deployment for multi-document transactions.
- If seat counts are stored as counters, reconcile them with enrolment states in tests or a maintenance check.
- State changes should record a reason and previous/current state where appropriate.

5. Proposed REST API contract

Method	Path	Access	Purpose
POST	/api/auth/login	Public, rate-limited	Authenticate; establish session.
POST	/api/auth/logout	Authenticated	Invalidate current session.
GET	/api/auth/me	Authenticated	Current user role and scope, never secrets.
POST	/api/batches	College Operator	Upload a CSV; returns batchId and status.
GET	/api/batches	Institution-scoped / relevant reviewer	List visible batches with safe counts.
GET	/api/batches/:batchId	Batch-scope permission	Batch status and summary.
GET	/api/batches/:batchId/issues	Batch-scope permission	Row issues and resolution status.
PATCH	/api/batches/:batchId/rows/:rowNumber	College Operator, own staging batch	Save allowed field corrections.
POST	/api/batches/:batchId/revalidate	College Operator, own batch	Revalidate changed rows.
POST	/api/batches/:batchId/return	College Head, own institution	Return for correction with reason.
POST	/api/batches/:batchId/certify	College Head, own institution	Recheck blockers and commit safely; conflict returns no partial commit.
GET	/api/institutions/:id/seat-ledger?academicYear=YYYY-YY	Assigned institution/officer scope	Read bucket capacity/filled/remaining plus policy version.
GET	/api/enrolments	Scoped roles	List permitted enrolment records, masked by default.
PATCH	/api/enrolments/:id/state	Authorised institution/head/case action	Validated state transition with reason.
GET	/api/conflicts	Scoped institutions/officers	List visible open/resolved cases.
GET	/api/conflicts/:id	Visible case actors	Case details and timeline.
POST	/api/conflicts/:id/resolve	Authorised case actor	Apply permitted resolution and record reason.
POST	/api/conflicts/:id/escalate	Authorised officer	Escalation event.
GET	/api/dashboard/college	College Operator/Head	Own institution summaries.
GET	/api/dashboard/aicte/national	National Officer	National aggregate view.
GET	/api/dashboard/aicte/region	Regional Officer	Only assigned region aggregate view.
GET	/api/admin/users	System Admin	List managed users with no password hashes.
POST	/api/admin/users	System Admin	Create account under permitted role/scope.
PATCH	/api/admin/users/:id	System Admin	Change allowed status/scope with audit.
POST	/api/admin/seat-policies	Admin policy maker	Create draft rule version.
POST	/api/admin/seat-policies/:id/validate	Authorised policy actor	Validate draft config.
POST	/api/admin/seat-policies/:id/publish	Different authorised approver	Maker-checker publish only after approval.
POST	/api/verification-requests	Verifier	Create request: studentRef, purpose, fields, expiry.
GET	/api/verification-requests/:id	Verifier owner / Student owner	Read only the appropriate side of the request.
POST	/api/consents/:id/approve	Student owner	Approve displayed permitted fields.
POST	/api/consents/:id/deny	Student owner	Deny pending request.

Method	Path	Access	Purpose
POST	/api/consents/:id/revoke	Student owner	Revoke active consent.
POST	/api/credentials	Authorised issuer workflow	Create a prototype credential/QR reference.
GET	/api/verify/:credentialRef	Public minimal endpoint	Return minimal demo status without open student search.
POST	/api/credentials/:id/revoke	Authorised issuer or explicit authority	Revoke permitted credential; record actor/reason.
GET	/api/audit	Scoped audit reader	Read permitted audit events.
POST	/api/audit/verify-chain	Authorised officer/admin	Run integrity check and return evidence/status.

6. Standard API behaviour

- Return JSON with stable error codes and safe messages. Never return stack traces to users.
- Recommended HTTP semantics: 400 malformed request; 401 unauthenticated; 403 insufficient permission; 404 unknown or deliberately concealed out-of-scope resource; 409 state/capacity conflict; 413 upload too large; 422 valid syntax but invalid business fields; 429 rate limit; 500 unexpected server error.
- Use pagination for list endpoints; default limits must be bounded.
- Never accept institutionId, regionId, user role or owner reference from the browser as authoritative scope. Derive it from the session and stored role.
- Mutating endpoints should be idempotent where retries could duplicate a commit, consent decision or credential issuance. Document retry semantics.

7. API response examples

Upload accepted (illustrative)

```
{
  "batchId": "BATCH-DEMO-001",
  "status": "VALIDATING",
  "receivedRows": 100
}
```

Validation issue (illustrative)

```
{
  "rowNumber": 12,
  "ruleCode": "L1_REQUIRED_FIELD",
  "severity": "ERROR",
  "message": "Required field is missing",
  "field": "courseCode",
  "resolutionStatus": "OPEN"
}
```

8. Contract discipline

- Before UI implementation, agree on request and response shapes.
- Keep route/controller logic thin; put business rules in service functions testable without React.
- Do not let the AI rename fields independently across client, API and database.
- Any contract change requires updating this document, tests and all call sites.

AICTE VERIFIED STUDENT REGISTER

Validation, Seat Ledger and Conflict Rules

Deterministic validation contract, severity handling, capacity invariants, enrolment lifecycle and test examples.

Document: 07-RULES Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Core processing rule

Golden rule
Uploaded rows are staging data. A batch can become official only after required validation, correction/review, institution-head certification and a safe commit. A risk flag alone must not block a record unless an independently defined deterministic rule requires that outcome.

2. Seven validation layers

Layer	What is checked	Severity / outcome	Notes for implementation
L1 Schema	Required fields, types, enums, dates, allowed value shapes.	ERROR for missing/invalid mandatory values.	Use one server-authoritative validation service; browser checks are convenience only.
L2 External identity / APAAR mock	Identifier format and mock match status; name/DOB match only where demo fixture supports it.	Invalid demo format = ERROR; mismatch = WARNING or explicit state; external service down/opt-out = PENDING.	Do not call live registry; name matching threshold and fields must be documented if used.
L3 Duplicate in batch	Same student repeated in same batch/institution/course-year according to agreed duplicate key.	ERROR on duplicate row; preserve source row references.	Do not merge records silently.
L4 Seat ledger	Capacity for course/year/bucket from approved demo configuration.	ERROR if reservation would exceed configured capacity.	Capacity must be sourced from stored configuration; no guessed real quota percentages.
L5 Cross-college	Same student has apparently active enrolments at two institutions in the same academic year.	WARNING + conflict case.	False positives are possible; transfer/lifecycle resolution and authorised review are required.
L6 Consistency	Fields and admission conditions agree with the configured rules.	ERROR or WARNING according to explicit rule definition.	Examples: course/year conflict, invalid lifecycle transition, nationality flag conflict; do not assert unverified policy.
L7 Risk signals	Batch/institution patterns such as late additions, repeated contact details, unusual ratios.	RISK_FLAG only; does not by itself block or prove wrongdoing.	Record signal reasons and ruleset version; false-positive feedback must not erase original evidence.

3. Severity definitions

Severity	Meaning	Can batch proceed?	Expected UI wording
ERROR	A deterministic required condition failed.	No certification/commit while blocking error is unresolved.	Explain field/row and how to correct it.
WARNING	Data may be valid but needs review or justification.	Only according to explicit warning-disposition policy. Never silently ignore.	Explain why it is a warning and what review is required.
PENDING	A check cannot yet complete or is in a permitted pending state.	Depends on explicit policy; default is do not imply verified.	State which check is pending and retry/alternate process.
INFO	Non-blocking informational outcome.	Yes, unless another rule blocks.	Present as contextual information.
RISK_FLAG	Pattern deserves investigation.	Does not automatically block.	Show signal, evidence and review status; no guilt language.

4. Illustrative synthetic data cases

Case	Input condition	Expected result
Missing field	courseCode empty	L1 ERROR; certification blocked until corrected.
Bad identifier format	Mock identifier contains 11 digits where the demo format requires 12	L2 format ERROR; this checks format only, not real-world identity.
Duplicate row	Same demo studentRef repeated within same batch/course/year	L3 ERROR with both row numbers.
NRI over capacity	Configured capacity is 3; four NRI seats would be reserved	L4 ERROR and no overflow commit.
Valid additional seats	Total exceeds approved intake due to explicitly configured additional buckets, and every bucket remains within configured limits	Do not trigger generic total-only overflow; accept capacity only if configuration/approval authorises those buckets.
Transfer not recorded	Student has an active record in two colleges in same year	L5 WARNING and conflict case; no automatic fraud verdict.
Registry unavailable	Mock adapter returns unavailable	PENDING/retry state, not fake verification success.
Repeated phone	Many synthetic students share a phone field	L7 signal and evidence; human review.
Correction made	Operator fixes one validation field	Revalidate relevant row; keep a record of resolution.
Rule changed after validation	Seat-policy version changes before certification	Revalidate or reject stale batch under documented version policy.

5. Seat ledger model

Suggested unique ledger identity: course/scope + academicYear + bucketCode. Store capacity, filled count, effective policy version and update timestamps. Every capacity must carry provenance: demo fixture ID or authorised approval reference. In a prototype, capacity can be deliberately fictional but must be labelled as such.

Config example - no real quota values

```
{
  "academicYear": "2026-27",
  "policyVersion": "DEMO-v1",
  "sourceType": "SYNTHETIC_DEMO_CONFIG",
  "buckets": [
    { "code": "REGULAR", "capacity": 57, "countsWithinIntake": true },
    { "code": "NRI", "capacity": 3, "countsWithinIntake": true },
    { "code": "TFW", "capacity": 3, "countsWithinIntake": false },
    { "code": "INTERNATIONAL", "capacity": 3, "countsWithinIntake": false }
  ]
}
```

These capacities are example test data only. They are not official policy and must never become production defaults. Keep the sourceType visible in the configuration and UI.

6. Commit and transaction invariants

- Certification service must re-check current batch status and blockers.
- Seat checks and seat-counter updates must be atomic with enrolment writes in the supported commit workflow.
- If any required write fails, abort/rollback the complete commit.
- Use a MongoDB replica set or sharded cluster to exercise multi-document transactions.
- Prevent duplicate certification and duplicate retries from committing a batch more than once.
- Use an idempotent state transition and explicit transaction outcome.

- Add an automated reconciliation test/check comparing ledger filled counts to relevant enrolment states.
- A counter must never be allowed to become negative or exceed capacity.

7. Enrolment lifecycle

State	Meaning in prototype	Seat effect (must be explicit)
STAGED	Row uploaded but not official.	No official seat counted.
PROVISIONAL	Validated/certified reservation state, as defined by approved workflow.	Reserve seat if the project rules say provisional enrolments occupy capacity.
CONFIRMED	Confirmed enrolment.	Counts as occupied under the configured rules.
WITHDRAWN	Student no longer occupies the admission.	Release seat once, with history.
TRANSFERRED	Enrolment moved to another institution/programme.	Release old scope only when the transition is accepted; track destination case.
COMPLETED	Programme completion.	Do not treat as current intake occupancy; retain historical record.
DROPPED	Enrolment ended for documented reason.	Release according to policy; keep history.

This is a proposed state contract aligned to the lifecycle concepts in the source report. Exact transitions and seat effects must be encoded in one transition table and tested; do not allow arbitrary direct status changes.

8. Conflict workflow

1. Detect candidate conflict from studentRef + academic year + enrolment state.
2. Create a Conflict record with both enrolment references and safe display fields.
3. Notify responsible institution users and authorised regional/national officers.
4. Allow institution to submit the supported resolution (withdrawn/transferred/justified) with a reason.
5. Run the state transition and seat release/reserve consistently; append history and audit.
6. Close only when the configured resolution conditions are satisfied; otherwise keep open/escalate.

9. Required tests

- Every validation layer has at least one passing and one failing test.
- Duplicate detection reports both relevant row numbers and does not merge silently.
- Configured bucket capacity 3 rejects a fourth reservation.
- Allowed additional buckets do not trigger an incorrect total-only overflow.
- Concurrent final-seat reservation yields at most one success.
- A failed transaction leaves neither official enrolment nor filled counter incremented.
- Seat release on withdrawal happens exactly once.
- Transfer creates/reconciles conflict and preserves historical state.
- Risk flag does not block or punish automatically.
- Policy-version mismatch forces revalidation or an explicit reject path.

AICTE VERIFIED STUDENT REGISTER

Security, Privacy, Testing and Acceptance Criteria

Practical safety controls and evidence required before describing the prototype as demo-ready.

Document: 08-SEC-QA Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Security posture and intended use

This is a learning/demo prototype using synthetic data. It must not be used to store real government/student records or presented as production-ready until a separate security, legal, operations and integration review is completed.

2. Minimum control requirements

Area	Required control	Evidence to collect
Passwords	Use a suitable password hash; no plaintext or reversible password storage.	Inspect model, seed scripts and logs; test wrong-password response.
Sessions	HttpOnly cookie, suitable SameSite, Secure under HTTPS, expiration, session regeneration and server-side invalidation.	Test login, logout, expiry and disabled account.
CSRF	Protect cookie-authenticated state-changing requests.	Document chosen maintained mechanism and test rejection of missing/invalid token.
Authorization	Backend checks role and institution/region/student scope on every protected route.	Automated 401/403/cross-scope tests.
Public endpoints	Rate-limit login and QR verification; no open student-search endpoint.	Test repeated requests and anonymous QR output.
CSV uploads	Allow-list file types, size/row bounds, safe parser, safe temp storage and cleanup.	Test malformed/oversized file and path/name manipulation.
Secrets	Use .env in local development, provide .env.example without values, exclude .env from Git.	Secret scan/manual repository review.
Sensitive data	Use synthetic identifiers; mask fields in UI/logs/exports.	Inspect API output and logs for raw IDs/credentials.
Database	Use least-privilege credentials; transaction-capable deployment for commit; validate indexes.	Run migration/index checks and failure/rollback tests.
Audit	Append-oriented application flow, previousHash/hash checks, restricted access and reasoned events.	Alter fixture in an isolated test and confirm detection.
Errors	Do not return stack traces, passwords, session IDs or raw internal exception objects.	Force invalid and server-error cases.
Dependencies	Use locked dependency versions and inspect vulnerability/audit output before demo.	Save command output and document unresolved advisories.

3. Data-minimisation rules

- Use fully synthetic student data generated for the demo. Use fake names and artificial IDs clearly marked as demo values.
- Do not include real APAAR or Aadhaar numbers, real DOBs, production credentials or private government documents in source control, prompts, screenshots or logs.
- Store only fields needed for the demo workflow. Do not collect phone/email unless a stated synthetic scenario requires it.
- Public QR result must not permit enumerating students by sequential IDs. Use a random opaque credential reference.
- Consent must bind to purpose, fields and expiry; server checks the grant on every read, and revocation blocks later access.
- A consent UI is not a legal compliance certification. Confirm applicable privacy/legal requirements with a qualified owner before real deployment.

4. Audit-chain integrity: honest boundaries

A hash chain can reveal an accidental or uncoordinated change that breaks the chain. It does not make MongoDB immutable, and it is not sufficient protection against an attacker who can rewrite all relevant records and control the verification keys/checkpoints. The demo should claim only that a specific alteration test was detected. Production protection requires separately protected signing keys/checkpoints and independent controls.

5. QR and credential boundaries

- URL-based QR: enables lookup/status demonstration; it does not by itself prove issuer authenticity.
- Mock status endpoint: proves only the demo app state, not real government records.
- Cryptographically signed credential: deferred unless a reviewed signing/verification design and protected key handling are implemented.
- Revocation status: the verifier must check current persisted demo status; a screenshot or old cached status is not proof of current validity.

6. Required automated tests

Test group	Minimum cases	Pass condition
Auth	Valid/invalid login; disabled user; expiry; logout; missing CSRF.	Only valid, active and protected requests proceed.
Authorization	Operator tries certify; College A requests College B; regional officer accesses outside region; student accesses other student's consent.	All unauthorised attempts fail and do not mutate data.
Upload	Bad MIME/extension, malformed CSV, missing columns, large file, duplicate rows.	Safe rejection or row issues; no official writes before commit.
Validation	One test per L1-L7, plus warning/pending semantics.	Expected issue code/severity and batch outcome.
Seat transaction	At capacity, over capacity, concurrent last seat, forced transaction failure.	No over-capacity state; no partial mutation.
Batch workflow	Certify with errors; return for correction; double-submit; stale policy.	State transitions follow contract and remain idempotent.
Conflict	Transfer, unresolved, justified case, double resolution.	Correct case status/history/seat effects.
Consent	Approve, deny, expiry, revoke, wrong student, field tampering.	Only current, approved fields returned.
QR	Valid demo, revoked demo, malformed/altered reference, anonymous enumeration attempt.	Minimal status response and correct state.
Audit	Normal chain; modify one stored event; access restricted.	Integrity mismatch reported without false success.
UI	Loading, empty, error, keyboard navigation, clear labels.	No dead buttons; actions have explicit feedback.
Setup	Fresh clone, env setup, DB setup, seed, run and tests.	Another developer can follow README with no hidden secrets.

7. Release/demo acceptance gate

- All critical flows in document 01 work end-to-end with seeded synthetic data.
- All role buttons in document 04 map to APIs and permissions; no fake/dead controls are shown.
- Every protected API has positive and negative permission tests.
- Seat ledger invariants pass under concurrency and rollback tests.
- Mock external adapters are labelled in code, UI and demo narration.
- Unverified performance targets are called targets, not measured achievements.
- README explains MongoDB topology, seed accounts, environment variables, startup and tests.
- Known issues and out-of-scope production requirements are stated in the demo materials.

8. Never make these claims without evidence

- “Connected to real APAAR/DigiLocker/AICTE APIs.”
- “Fraud detection accuracy is X%” when only synthetic anomalies were planted.
- “100% tamper-proof,” “immutable,” or “blockchain-equivalent.”
- “Handles lakhs of records in production” without reproducible load tests and deployment specifications.
- “Fully compliant with all data-protection law” without a legal/compliance review.
- “Signed credential” when QR only contains a URL or ordinary identifier.

AICTE VERIFIED STUDENT REGISTER

Vibecoding Prompt Pack and Build Order

Copy-ready prompts to run a disciplined build with an AI coding agent, from plan to final QA.

Document: 09-PROMPTS Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Operating model

Use one coding agent as the primary editor. Give it one milestone, its source PDFs and acceptance criteria. Do not ask it to implement the whole 33-page system in one request. After each milestone, inspect diffs, run the app, execute tests and create a Git checkpoint.

2. Prompt 0 - read and plan only

```
Read these source attachments before coding:
- 01_Problem_Statement_and_Product_Scope.pdf
- 02_Tech_Stack_and_Repository_Spec.pdf
- 05_System_Architecture_and_Workflows.pdf

Do not modify or create application files yet. Return: requirements summary,
architecture, role/permission map, collection model, API contracts, milestone
plan, test plan, risks and an explicit TBD list. Mark each statement as source
requirement, proposed implementation or unresolved. Do not invent official seat
quotas, real API access, credentials, legal requirements or measured performance.
Wait for review before coding.
```

3. Prompt 1 - foundation

```
Implement only Phase 1 from the approved plan. Use the exact stack in
02_Tech_Stack_and_Repository_Spec.pdf. Inspect the existing repository first.
Create React/JavaScript/Bootstrap client, Express server, environment example,
MongoDB connection, health check, error handler, README and basic tests.
Do not implement admission business logic yet. Run checks and report the exact
commands/results, changed files, new packages, assumptions and remaining issues.
```

4. Prompt 2 - authentication

```
Attach 03_Authentication_and_Authorization_Spec.pdf and 04_Role_Features_and_Button_Inventory.pdf.
Implement login/logout, server-side sessions, password hashing, current-user
endpoint and backend RBAC/scope checks. Use only seeded synthetic accounts.
Do not proceed to CSV uploads. Test invalid credentials, 401, 403, cross-college
access, regional scope, disabled account and logout invalidation. Report failures honestly.
```

5. Prompt 3 - upload and validation

```
Attach 06_Database_and_API_Contract.pdf, 07_Validation_Seat_Ledger_and_Conflicts.pdf,
and 08_Security_Privacy_Testing_and_Acceptance.pdf. Implement CSV ingestion,
batch/staging records, row-level issues, correction and revalidation only. Keep
all staged data out of the official register. Use mock external adapters. Test bad
file, missing fields, invalid demo ID format, duplicate rows and upload limits.
```

6. Prompt 4 - seat ledger and certification

```
Implement only the seat ledger and maker-checker commit in document 07. Use
MongoDB transactions where required and document/configure a replica set or
supported hosted topology. Demo capacities must come from synthetic config, not
real policy defaults. The commit must recheck blockers and policy version; update
seat counter, enrolment and audit/history consistently; rollback on failure. Test
capacity overflow, concurrent last-seat attempts and forced transaction failure.
```

7. Prompt 5 - conflict and dashboards

```
Attach 04, 05, 06, 07 and 08. Implement cross-college conflict review and the
role-specific dashboards using real data from the local database. Preserve role
scopes on the server. Do not use hard-coded dashboard metrics. A cross-college
match is a case for review, not a fraud finding. Test transfer, unresolved case,
role violations and region/institution data isolation.
```

8. Prompt 6 - risk, consent, QR and audit

```
Implement one capability at a time using documents 04, 06, 07 and 08. First
rule-based risk flags with reasons; second consent approval/denial/revocation;
third mock QR status; fourth hash-chain audit integrity. Never call the rule-based
score machine learning, an unsigned QR a signed credential, or a mock adapter a
real government integration. Add tests for false positives, expiry, revocation,
malformed QR references and modified audit entries.
```

9. Prompt 7 - independent QA

```
Attach 08_Security_Privacy_Testing_and_Acceptance.pdf. Do not add features.
Inspect code, run tests and report pass/fail/not run with exact commands. Check
role authorization on the server, data scope, transaction safety, upload limits,
secret handling, consent expiry/revocation, QR boundaries and audit integrity.
Fix confirmed defects only, rerun affected tests, and provide residual risks. Do
not claim production readiness or report unexecuted tests as passed.
```

10. Debugging prompt

```
Observed behaviour:
[What happened]
Expected behaviour:
[What should happen]
Exact command and error:
[Paste output]

Inspect the existing code and identify the root cause before editing. Explain the
responsible file/function, smallest safe fix and likely regression risks. Apply only
the minimal correction, run relevant tests and report the real result. Do not
rewrite unrelated files, change the stack or suppress the error.
```

11. Review prompt after any change

```
Review the diff for this milestone against its PDF specification. Identify
requirements not met, dead buttons, APIs with no UI or UI with no API, missing
authorization, inconsistent fields, unhandled errors, untested code and scope drift.
Do not add new features. Present findings by severity and cite the specific file,
route or function. Fix only confirmed defects after I approve the patch.
```

12. Per-milestone report the agent must return

Report item	Required detail
Summary	What is implemented and what remains unimplemented.
Changed files	Path and purpose for each material change.
Dependencies	New package, reason, environment/config changes.
Commands	Exact tests/build/start commands executed.
Test results	Pass/fail/not run; include failure summaries.
Assumptions/TBD	Anything not supported by source or not decided.
Security / privacy	Relevant protections and what is not yet protected.
Next step	The single next milestone; no unapproved scope expansion.

13. Git checkpoint discipline

1. Before each milestone, run git status and create a commit/branch for the known-good state.
2. Review the diff after the agent edits; reject unexplained wholesale rewrites.
3. Run the milestone tests yourself or inspect the actual captured output.
4. Commit only after acceptance criteria pass.
5. If a change breaks a known-good milestone, revert or fix the narrow diff before moving forward.

14. Final anti-hallucination check

- Ask the agent to name the PDF and section supporting each product requirement.
- Require it to label its own engineering choices as proposed when they are not in source.
- Ask for proof of live integration before accepting any claim about external APIs.
- Ask for measured test evidence before accepting performance or accuracy numbers.
- Verify that seat rules have a source reference/version or are labelled synthetic configuration.
- Verify that all role buttons are listed, wired, permission-checked and tested.

AICTE VERIFIED STUDENT REGISTER

Synthetic Demo Data and Test Scenarios

Safe fake data conventions, CSV shape, role seed accounts and repeatable end-to-end demonstrations.

Document: 10-DEMO-DATA Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Why this document exists

The coding agent needs repeatable data to build and test each screen. This specification defines the shape and meaning of fake demo records. It does not contain real students, real government IDs, real accounts or official quota limits.

No real personal data
All names, identifiers, phone tokens, colleges and records in this specification are fabricated for software testing. Never replace them with real APAAR/Aadhaar values or student records.

2. CSV upload contract

Column	Required?	Meaning / validation
studentCode	Yes	Synthetic local code such as STU-001. It is a demo reference, not an official AICTE identifier.
studentName	Yes	Synthetic display name; trim whitespace; empty name is L1 ERROR.
courseCode	Yes	Must match a course configured for the authenticated institution and academic year.
academicYear	Yes	Must match the batch academic year and approved demo configuration.
bucketCode	Yes	Must match a configured bucket code. The template must derive permitted values from active demo config.
externalIdDemo	Required only when the mock identity check is configured for this test	Fake external identifier for mock-only format checking. A 12-digit format fixture mirrors the report’s example but does not validate a real identity.
contactToken	No, used in selected risk fixtures	Synthetic token such as PHONE-DEMO-A; never a real phone number.
yearOfStudy	Yes if the configured rule set requires it	Integer or permitted enum; exact constraints must come from the selected demo rule set.
admittedOn	Yes if the configured rule set requires it	ISO date YYYY-MM-DD; must be consistent with demo cutoff configuration.

The institutionId and submitting user must come from the authenticated session and stored user record. Do not accept them from a CSV as trusted authority. The client may perform pre-checks, but server validation is authoritative.

3. Clean sample CSV

Save the following as clean_batch.csv for the first successful workflow. The values are fabricated. Keep the selected seat capacities high enough for this clean fixture.

```
studentCode,studentName,courseCode,academicYear,bucketCode,externalIdDemo,contactToken,yearOfStudy,admittedOn
STU-001,Aarav Sharma,CSE,2026-27,REGULAR,111111111111,PHONE-DEMO-A,1,2026-08-01
STU-002,Neha Verma,CSE,2026-27,REGULAR,222222222222,PHONE-DEMO-B,1,2026-08-02
STU-003,Kabir Singh,CSE,2026-27,REGULAR,444444444444,PHONE-DEMO-C,1,2026-08-03
STU-004,Meera Das,CSE,2026-27,REGULAR,555555555555,PHONE-DEMO-D,1,2026-08-04
```

4. Negative fixtures

Fixture	Constructed condition	Expected system behaviour
missing_required_field.csv	A row has an empty courseCode or studentName.	Create L1 ERROR with row number and field name; do not commit.
bad_mock_external_id.csv	externalIdDemo is 11 digits while the selected demo format rule requires 12.	Return mock-format L2 ERROR. Do not claim a real registry lookup occurred.
duplicate_in_batch.csv	The same studentCode/externalIdDemo and course/year are repeated in a single batch.	L3 ERROR references both row numbers; do not merge or delete silently.
bucket_overflow.csv	Demo rule config capacity for NRI is 3; fixture tries to reserve 4 NRI seats.	L4 ERROR; no partial commit and no seat counter above 3.
registry_pending.csv	Mock adapter is configured to return SERVICE_UNAVAILABLE/PENDING.	Set the identity check to PENDING; never report verified; batch decision follows explicit pending policy.
cross_college_conflict.json	STU-005 has apparent active enrolments at College A and College B for the same academic year.	Create L5 conflict case; leave case open until an authorised state transition/justification resolves it.
shared_contact_risk.json	Twelve synthetic student records share contactToken PHONE-DEMO-SHARED.	Create an L7 risk signal with count/evidence; do not block solely for this signal or label fraud.
stale_policy_batch.json	Batch was validated under DEMO-v1; current active demo rules become DEMO-v2 before certification.	Revalidate or reject according to the documented policy-version rule; never silently use mixed capacities.

5. Seat-ledger demo configuration

Use a separate synthetic configuration file and display its demo label in the admin UI. The following capacities are for a deterministic test only; do not ship them as actual AICTE limits.

```
{
  "academicYear": "2026-27",
  "policyVersion": "DEMO-v1",
  "sourceType": "SYNTHETIC_DEMO_CONFIG",
  "institutionCode": "ABC-DEMO",
  "courseCode": "CSE",
  "buckets": [
    { "code": "REGULAR", "capacity": 57, "countsWithinIntake": true },
    { "code": "NRI", "capacity": 3, "countsWithinIntake": true },
    { "code": "TFW", "capacity": 3, "countsWithinIntake": false },
    { "code": "INTERNATIONAL", "capacity": 3, "countsWithinIntake": false }
  ]
}
```

6. Seeded demo accounts

Create one synthetic account per role. Use reserved .test email addresses and synthetic institution/region references. Do not place fixed working passwords in this PDF, source code, or version control. The seed script should require locally supplied development secrets or generate one-time random passwords and display them only in the developer terminal.

Account alias	Role	Assigned scope in demo
operator.demo@example.test	COLLEGE_OPERATOR	ABC-DEMO institution
head.demo@example.test	COLLEGE_HEAD	ABC-DEMO institution
national.demo@example.test	AICTE_NATIONAL_OFFICER	National aggregate scope
regional.demo@example.test	AICTE_REGIONAL_OFFICER	REGION-DEMO only
admin.demo@example.test	AICTE_SYSTEM_ADMIN	System user/config management
issuer.demo@example.test	DEGREE_AWARDING_BODY	ISSUER-DEMO scope only
verifier.demo@example.test	VERIFIER	VERIFIER-DEMO organisation

Account alias	Role	Assigned scope in demo
student.demo@example.test	STUDENT	STU-001 only

Seed script acceptance: refuses to run in production mode; hashes passwords; does not print hashes; is safe to rerun (upserts predictable demo data without duplicating users); creates no real data; warns that accounts are demo-only.

7. Repeatable seven-minute demo flow

Step	Action	Expected outcome
1. Upload	Login as operator and upload clean/negative fixture.	Batch receives ID/status; progress and row count appear.
2. Fix errors	Open issue grid, correct allowed fields and revalidate.	Blocking issues clear only when inputs actually satisfy rules.
3. Seat ledger	Run capacity fixture using DEMO-v1 config.	Within-capacity categories pass; an NRI capacity of 3 rejects a fourth reservation.
4. Conflict	Open the STU-005 conflict fixture.	Both authorised college sides see appropriate case; transition resolves only after valid action.
5. Risk queue	Open shared-contact risk signal.	Score/reasons/evidence appear; a reviewer can mark reviewed or false positive.
6. Consent	Login as verifier, submit request, then as student approve it.	Verifier sees only approved fields and a recorded access event.
7. QR + audit	Show demo QR status, revoke it, then run audit-chain test.	Status changes to revoked; controlled audit alteration fixture is detected.

8. Test fixture hygiene

- Keep fixtures deterministic: fixed synthetic IDs, dates and rule versions. Do not use random values that make expected test outputs unstable.
- Use separate fixtures for different failure modes so one early blocking error does not mask another test.
- Reset demo data only through a documented development-only reset command; never add a public reset endpoint.
- Mark all UI labels and export filenames as DEMO where confusion with real credentials is possible.
- Capture the exact configuration, command and test output when making any performance statement.

9. Open decisions

- The owner must approve exact column names before the first implementation contract is frozen.
- A real identity registry contract is not included; mock outcomes are selected by deterministic fixtures.
- Official seat rules remain TBD. DEMO-v1 capacities are synthetic tests only.
- Any legal basis for non-consensual verifier access is out of prototype scope; default test flow requires student approval.